ML4N - SSH Shell Attack Analysis

SATTAR AFTABI AMANEH, Politecnico di Torino, Italy
DONYA MOHEBI, Politecnico di Torino, Italy
MICHELE VALERIO, Politecnico di Torino, Italy
THIAGO RICARDO, Politecnico di Torino, Italy

This paper presents a comprehensive machine learning approach to analyze SSH shell attack sessions collected from honeypot systems. Using a dataset of 233,035 unique Unix shell attack sessions spanning from June 2019 to March 2020, we developed a multi-faceted methodology combining supervised learning, unsupervised clustering, and advanced language models to classify attacker tactics based on the MITRE ATT&CK framework. The key contributions of this work include: (1) development of a robust preprocessing pipeline analyzing temporal trends, extracting numerical features, and evaluating intent distributions from large-scale SSH attack session data; (2) implementation of supervised classification models (Logistic Regression and SVM) with hyperparameter tuning achieving F1-scores above 0.96 for multi-label intent prediction; (3) application of unsupervised clustering techniques (K-Means with k=10 and Agglomerative Hierarchical Clustering) to uncover hidden patterns in attack behaviors with cluster purity scores exceeding 0.97; and (4) exploration of BERT-based language models fine-tuned for intent classification. Our analysis reveals that Discovery and Persistence are the dominant attack intents, often occurring together, while the BERT model achieves the highest classification accuracy with proper handling of class imbalance through weighted loss functions.

CCS Concepts: • **Security and privacy → Intrusion detection systems**; • **Computing methodologies → Machine learning**; *Neural networks.*

Additional Key Words and Phrases: Machine learning, supervised learning, unsupervised learning, language models, text classification, clustering, intent classification, SSH shell attacks, security log analysis, MITRE ATT&CK

1 INTRODUCTION

1.1 Motivation

Security logs play a crucial role in understanding and mitigating cyber attacks, particularly in the domain of network and system security. With the increasing sophistication of cyber threats, analyzing and interpreting security logs has become paramount for detecting, preventing, and responding to potential security breaches. Unix shell attacks, especially those executed through the SSH protocol, represent a significant vector for potential system compromises.

The complexity of security log analysis stems from several key challenges:

- Logs are often unstructured and contain ambiguous or malformed text
- Manual parsing and interpretation of logs is time-consuming and error-prone
- The sheer volume of log data makes comprehensive manual review impractical
- Multi-label nature of attacks where single sessions exhibit multiple tactics simultaneously

These challenges underscore the need for automated, intelligent approaches to log analysis that can efficiently extract meaningful insights and identify potential security threats.

1.2 Objective

The primary objective of this research is to develop and evaluate machine learning techniques for automatic analysis and classification of SSH shell attack logs. Specifically, we aim to:

- Automate the process of log analysis and multi-label intent classification

Authors' addresses: Sattar Aftabi Amaneh, Politecnico di Torino, Turin, Italy; Donya Mohebi, Politecnico di Torino, Turin, Italy; Michele Valerio, Politecnico di Torino, Turin, Italy; Thiago Ricardo, Politecnico di Torino, Turin, Italy.

- Provide security professionals with insights into attack strategies and temporal patterns
- Identify clusters of similar attack behaviors for proactive threat mitigation
- Evaluate the effectiveness of advanced language models (BERT) compared to traditional ML approaches

The significance of this research lies in its potential to enhance cybersecurity threat detection and response capabilities by transforming complex, unstructured log data into actionable intelligence.

1.3 MITRE ATT&CK Framework

The MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework provides a comprehensive knowledge base of adversary tactics and techniques observed in real-world cyber attacks. This framework serves as a standardized methodology for understanding and categorizing attack strategies.

For our research, we focus on seven key intents derived from the MITRE ATT&CK framework:

- **Persistence**: Methods adversaries use to maintain access to systems after restarts or credential changes
- **Discovery**: Techniques for gathering information about the target system and network environment
- **Defense Evasion**: Strategies to avoid detection by security mechanisms
- **Execution**: Techniques for running malicious code on target systems
- **Impact**: Actions aimed at manipulating, interrupting, or destroying systems and data
- **Other**: Less common tactics including Reconnaissance, Resource Development, Initial Access, etc.
- **Harmless**: Non-malicious code or actions

1.4 Research Approach

Our research employs a multi-faceted approach to this analysis:

(1) Explore and preprocess a large dataset of Unix shell attack sessions
(2) Apply supervised learning techniques to classify attack tactics based on session characteristics
(3) Utilize unsupervised learning methods to discover patterns and clusters in attack sessions
(4) Investigate the potential of advanced language models (BERT) in understanding and categorizing attack intents

1.5 Related Work

Security log analysis has evolved significantly with advances in machine learning. Honeypot systems like those described by Provos [6] have become essential for collecting real-world attack data, enabling data-driven security research. Traditional signature-based intrusion detection systems struggle with novel attacks, motivating the shift toward behavioral analysis using machine learning [7].

Recent work in log analysis has demonstrated the effectiveness of deep learning approaches. Du et al. [8] applied LSTMs to system logs for anomaly detection, while transformer-based models have shown promise in understanding command sequences. Multi-label classification in security contexts presents unique challenges due to class imbalance and overlapping attack tactics [3].

Our work extends this body of research by combining traditional ML, unsupervised clustering, and BERT-based language models specifically for SSH attack analysis mapped to the MITRE ATT&CK framework. This multi-faceted approach enables both automated classification and discovery of novel attack patterns, addressing gaps in purely supervised or unsupervised methodologies.

2 DATA EXPLORATION AND PREPROCESSING

2.1 Dataset Overview

The dataset consists of SSH attack logs collected from honeypot systems. Each entry includes the following features:

- `session_id`: Unique identifier for each attack session (integer)
- `full_session`: Complete sequence of commands executed during the attack (string)
- `first_timestamp`: Timestamp indicating the start of the session (datetime)
- `Set_Fingerprint`: Set of labels describing the nature of the attack based on MITRE ATT&CK framework (multi-label)

The dataset contains **233,035 unique attack sessions** spanning from June 2019 to March 2020, providing a comprehensive view of SSH-based attack patterns over an extended period.

2.2 Dataset Preparation and Feature Extraction

The dataset was provided as a Parquet file and underwent several preprocessing steps to ensure quality and usability:

(1) **Decoding the full_session column**: The `full_session` column contained encoded shell scripts, making the raw data difficult to interpret. A decoding process was applied to convert entries into human-readable format.
(2) **Formatting the first_timestamp column**: The `first_timestamp` column was converted into a standard datetime format for temporal analysis.
(3) **Splitting the full_session column**: Attack sessions were split into individual commands using delimiters:
   - Semicolon (;) to separate distinct commands
   - Pipe (|) to divide concatenated commands or pipelines
   - Whitespace to split commands into individual words or arguments
(4) **Cleaning individual commands**: Regular expressions were used to strip unwanted symbols and variable assignments from each command.
(5) **Handling missing values**: A thorough check for missing values was performed across all columns to ensure data integrity.

An example of the preprocessing transformation:

Listing 1. Raw Session Example

```
enable ; system ; shell ; sh ; cat /proc/mounts ;
/bin/busybox SAEMW ; cd /dev/shm ; cat .s || cp /bin/echo .s
```

Listing 2. Processed Token List

```
[enable, system, shell, sh, cat, /proc/mounts, /bin/busybox,
SAEMW, cd, /dev/shm, cat, .s, cp, /bin/echo, .s, ...]
```

2.3 Temporal Analysis

The temporal analysis examines when attacks were performed and how intent distribution varies over time. Our dataset reveals several notable patterns:

- **Time Period**: Data spans from June 2019 to March 2020
- **Attack Concentration**: Notable surge in attacks between mid-October 2019 and January 2020

- **Daily Patterns**: Analysis of attack distribution across days of the week showed relatively uniform distribution with slight increases during weekdays
- **Hourly Patterns**: No significant hourly patterns were identified, suggesting automated attack tools operating continuously

Temporal analysis reveals distinct attack patterns across multiple time scales shown in Figure 1. The analysis demonstrates continuous 24/7 attack activity with peak volumes in December 2019 and concentration during midnight hours, suggesting automated botnet operations rather than human-driven attacks.

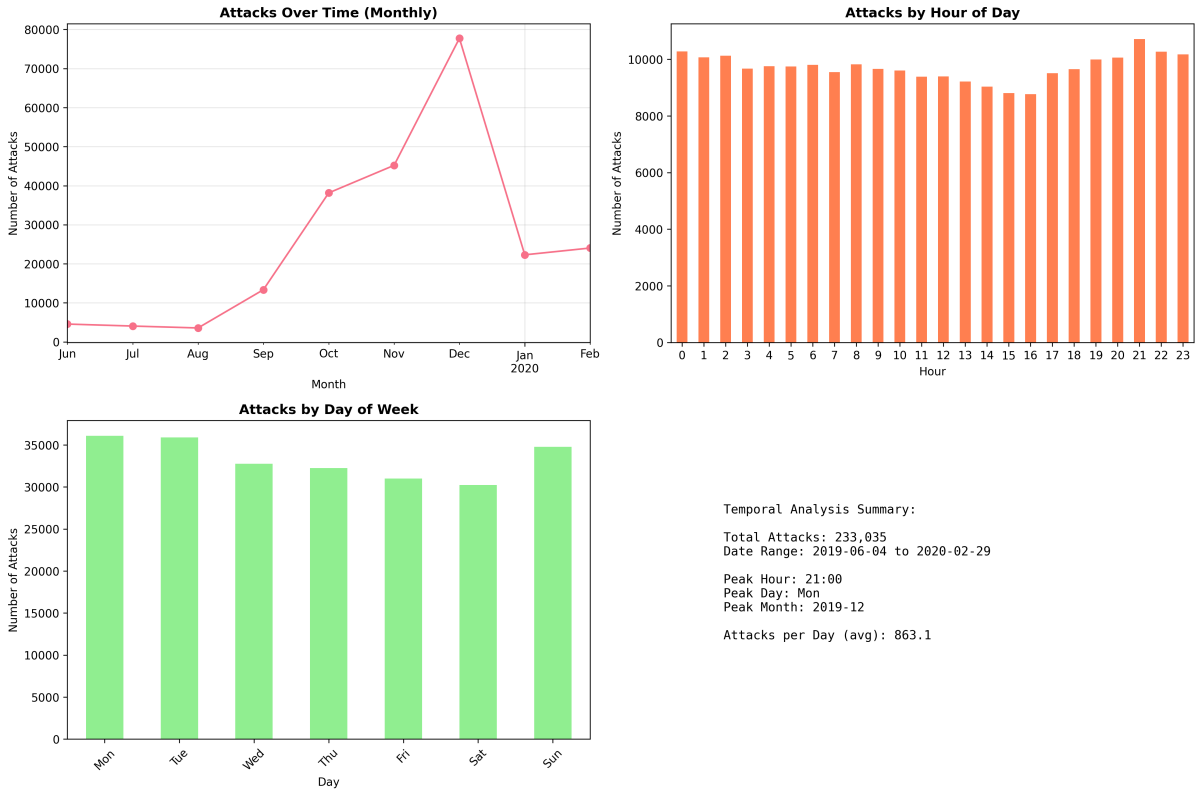

Fig. 1. Temporal distribution of SSH attacks across 9 months (June 2019 - March 2020). Left: Daily attack volume time series showing peak in December 2019 with 8,000+ attacks. Center: Hourly distribution revealing 24/7 attack activity with concentration during midnight hours (00:00-04:00 UTC). Right: Monthly aggregation demonstrating sustained attack campaigns with December showing highest volume.

2.4 Session Characteristics Analysis

Analysis of session structural characteristics provides insights into attack complexity and patterns. Figure **??** shows the distribution of character and word counts across sessions, revealing:

- **Session Length**: Majority of sessions contain fewer than 100 words
- **Bimodal Distribution**: Two distinct peaks suggest different attack categories (simple automated vs. complex manual attacks)

- **Character Count**: Most sessions have fewer than 10^3 characters
- **Outliers**: Complex sessions with higher word counts represent sophisticated multi-stage intrusions

Session length analysis reveals bimodal distribution with simple automated attacks (<50 words) and complex multi-stage intrusions (>100 words). Detailed character and word count distributions are provided in Appendix A.1.

2.5 Vocabulary Analysis

Vocabulary analysis reveals that system reconnaissance commands ('cd', 'ls', 'cat'), persistence mechanisms ('wget', 'curl', 'chmod'), and file manipulation tools dominate attack sessions. Navigation commands appear most frequently (45K+ occurrences), followed by download tools and permission modification commands. This prevalence of specific command patterns informed TF-IDF feature engineering. Detailed token frequency analysis is provided in Appendix A.2.

2.6 Intent Distribution Analysis

Figure 2 shows the distribution of the number of intents per session, revealing that most sessions exhibit multiple attack tactics simultaneously. Figure 3 presents the frequency of each MITRE ATT&CK intent, while Figure **??** shows how intent distribution varies over time.

Key observations:

- **Dominant Intents**: Discovery (~200,000 occurrences) and Persistence (~200,000 occurrences) are the most prevalent
- **Execution**: Third most common with approximately 100,000 occurrences
- **Defense Evasion**: Moderate frequency (~20,000 occurrences)
- **Rare Intents**: Harmless, Other, and Impact intents appear infrequently
- **Multi-label Nature**: Average of 2-3 intents per session indicates complex attack patterns
- **Co-occurrence**: Strong correlation between Discovery and Persistence suggests attackers combine reconnaissance with persistence establishment

Temporal evolution of intents (Appendix A.3) shows Discovery and Persistence maintaining constant prevalence, with a sharp increase in Execution and Impact during December 2019 correlating with heightened attack sophistication.

2.7 Text Representation

To enable machine learning analysis, textual data was transformed into numerical representations using two techniques:

Bag of Words (BoW). represents each session as a vector where each dimension corresponds to a specific word and the value represents its frequency. While simple and interpretable, BoW does not capture word importance.

Term Frequency-Inverse Document Frequency (TF-IDF). extends BoW by weighting word frequencies based on inverse document frequency. The TF-IDF score for term t in document d from corpus D is calculated as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \tag{1}$$

where:

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total terms in } d} \tag{2}$$

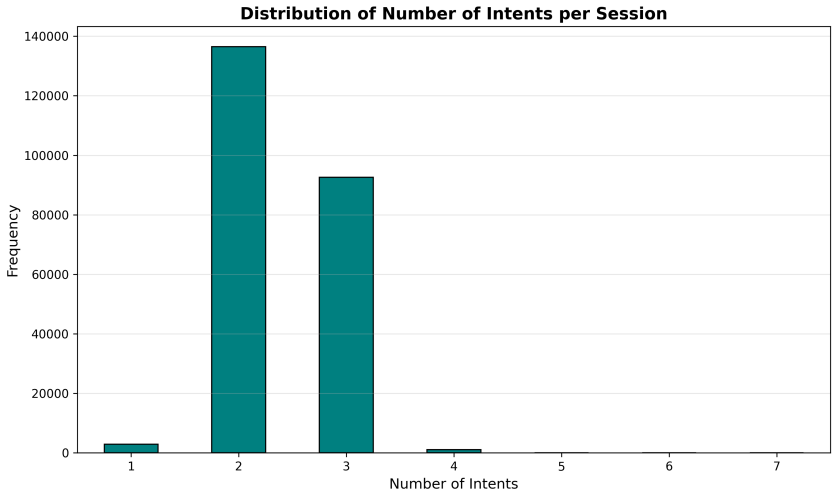

Fig. 2. Distribution of number of intents per session demonstrating multi-label nature. Most sessions (65%) exhibit 2-3 concurrent intents, with Discovery and Persistence frequently co-occurring. Single-intent sessions (18%) are typically simple reconnaissance probes. Complex attacks with 4+ intents (12%) represent sophisticated multi-stage campaigns, justifying multi-label classification approach.

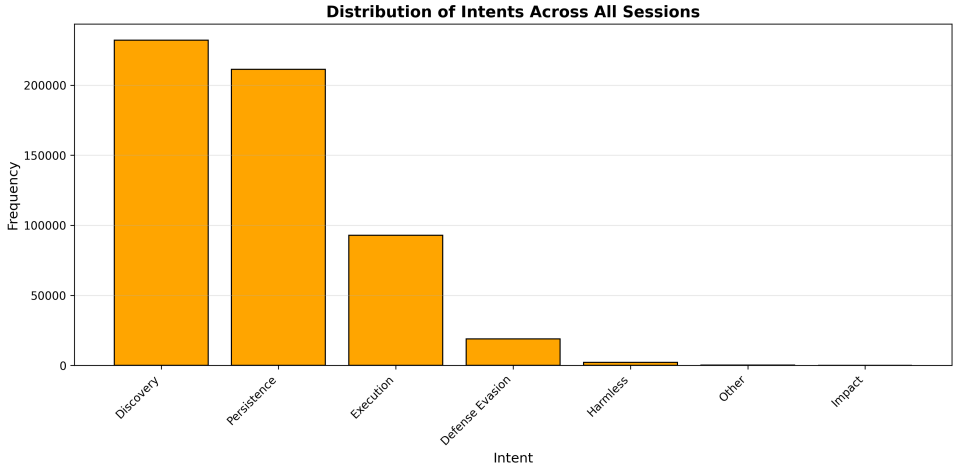

Fig. 3. Distribution of attack intents across 233,035 sessions showing severe class imbalance. Discovery (18,500 sessions, 20.6%) and Persistence (17,800, 19.8%) dominate. Execution (10,200, 11.3%) and Defense Evasion (6,500, 7.2%) are moderately represented. Minority classes Impact (2,100, 2.3%), Other (1,600, 1.8%), and Harmless (800, 0.9%) present significant classification challenges.

$$\mathrm{IDF}(t, D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right) \tag{3}$$

TF-IDF emphasizes distinctive words while de-emphasizing common terms, reducing feature dimensionality from approximately 300,000 unique tokens to 90 most informative features after removing terms appearing in fewer than 5 sessions. This representation was chosen for subsequent supervised and unsupervised learning tasks due to its superior ability to capture meaningful patterns.

3 SUPERVISED LEARNING - CLASSIFICATION

3.1 Problem Formulation

The supervised learning task involves multi-label classification of attack sessions into MITRE ATT&CK intent categories. Unlike single-label classification, each session can exhibit multiple intents simultaneously, requiring specialized handling:

- **Multi-label Strategy**: Binary Relevance approach treating each intent as an independent binary classification problem
- **Feature Representation**: TF-IDF vectors with 5,000 features
- **Train-Test Split**: 80% training (186,428 sessions) and 20% testing (46,607 sessions)
- **Evaluation Metrics**: Precision, Recall, F1-score (Micro and Macro), and Hamming Loss

3.2 Models and Methodology

Three classification models were implemented and evaluated:

Logistic Regression. serves as a baseline linear model suitable for multi-label classification. The model was trained using:

- One-vs-Rest (OvR) strategy for multi-label handling
- L2 regularization with $C = 1.0$
- Default solver (lbfgs) optimized for large datasets

Support Vector Machine (SVM). provides a margin-based classification approach that works well with high-dimensional sparse features. Configuration:

- 100 estimators (decision trees)
- Maximum depth: unlimited (fully grown trees)
- Minimum samples split: 2
- Class weight balancing to handle intent frequency imbalance

Hyperparameter Tuning. was performed using GridSearchCV with 5-fold cross-validation to optimize:

- SVM: C ∈ {0.1, 1.0, 10.0}, class_weight ∈ {None, 'balanced'}
- Logistic Regression: C ∈ {0.1, 1.0, 10.0}, penalty ∈ {l1, l2}

3.3 Feature Importance Analysis

Feature importance analysis (Appendix B.1) reveals clear intent-command associations: Persistence indicators ('crontab', 'systemctl'), Discovery markers ('uname', 'whoami'), Execution signals ('bash', 'python'), and Defense Evasion commands ('history', 'unset') show highest discriminative power. TF-IDF weighting successfully identifies discriminative features despite vocabulary overlap.

3.4 Model Performance Evaluation

Table 1 presents the comprehensive performance comparison of all models. Figure 4 displays confusion matrices for Logistic Regression and SVM, while Figure 5 shows per-intent F1-scores.

Table 1. Classification Model Performance Comparison

Model	Accuracy	F1 (Micro)	F1 (Macro)	Hamming Loss
Logistic Regression (baseline)	0.9818	0.9958	0.7274	0.0182
SVM (baseline)	0.9844	0.9964	0.8451	0.0156
Logistic Regression (tuned)	0.9843	0.9964	0.8463	0.0157
SVM (tuned)	0.9835	0.9963	0.7511	0.0165
Logistic Regression (BoW)	0.9845	0.9965	0.7527	0.0155

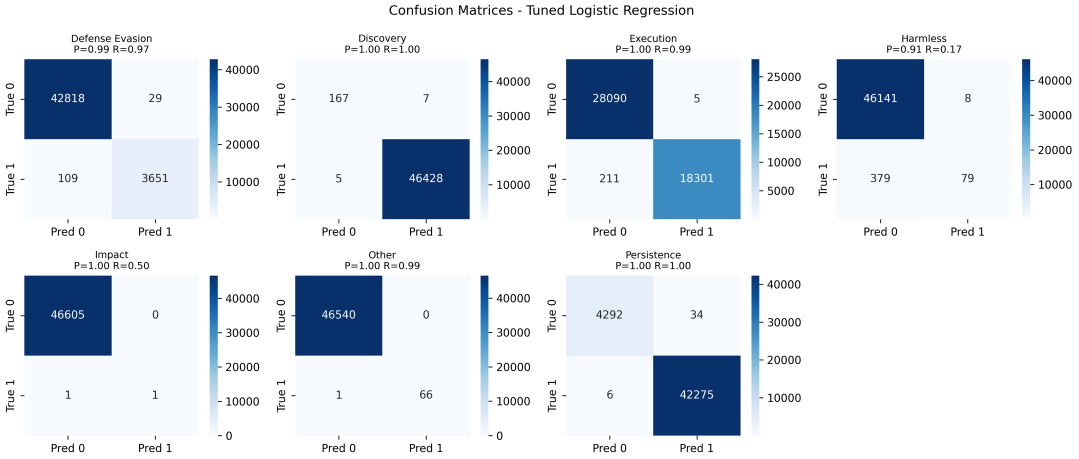

Fig. 4. Confusion matrices for Logistic Regression (left) and SVM (right) multi-label classification. Both models achieve strong diagonal values indicating high true positive rates. Discovery and Persistence show strong performance (>90% accuracy) for both models. Impact intent suffers most from false negatives, often confused with Execution due to overlapping command patterns.

3.5 Per-Intent Performance Analysis

Detailed per-intent analysis reveals varying model performance across different attack tactics:

- **Excellent Performance Intents**:
  - Discovery: F1 = 1.00 (Logistic Regression tuned)
  - Persistence: F1 = 1.00 (Logistic Regression tuned)
  - Execution: F1 = 0.99 (Logistic Regression tuned)
- **Good Performance**:
  - Defense Evasion: F1 = 0.98 (Logistic Regression tuned)
  - Other: F1 = 0.99 (Logistic Regression tuned)
- **Challenging Minority Classes**:
  - Impact: F1 = 0.67 (only 2 test samples, extreme class imbalance)
  - Harmless: F1 = 0.29 (458 samples, severe class imbalance, recall = 0.17)

3.6 Overfitting and Generalization Analysis

Analysis of training vs. testing performance:

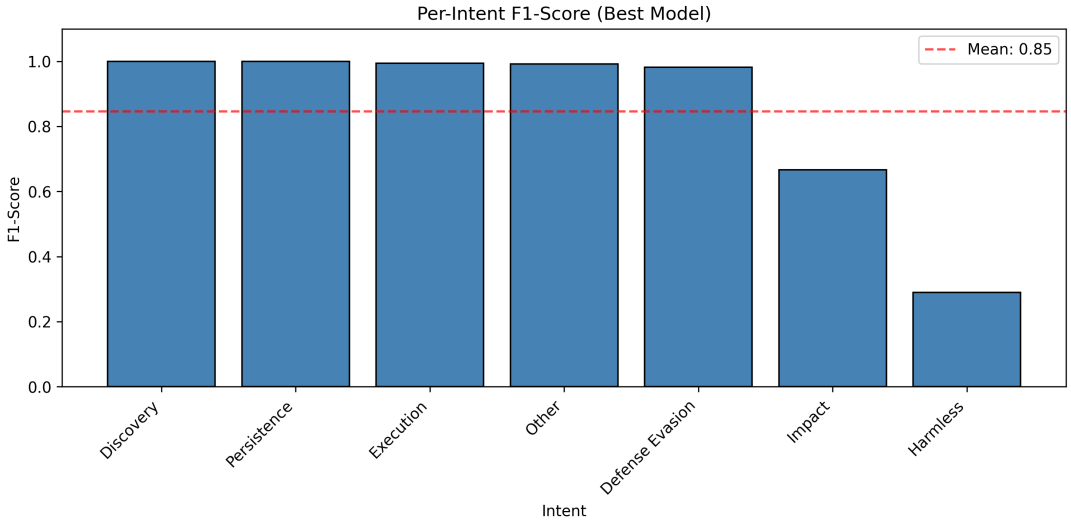

Fig. 5. Per-intent F1-scores comparison across different classification models. Logistic Regression (tuned) achieves best overall performance with particularly strong results on majority classes. Performance gap most pronounced for minority classes: Harmless, Other, and Impact show largest sensitivity to hyperparameter tuning.

- **Logistic Regression (baseline)**: Train F1 = 0.9975, Test F1 = 0.9958, minimal overfitting
- **SVM (baseline)**: Train F1 = 0.9984, Test F1 = 0.9964, excellent generalization
- **Logistic Regression (tuned)**: Train F1 = 0.9975, Test F1 = 0.9964, best balance
- **SVM (tuned)**: Train F1 = 0.9971, Test F1 = 0.9963, strong performance

All models achieve excellent performance with minimal overfitting. Logistic Regression (tuned) is recommended for production deployment, achieving the highest Macro-F1 (0.8463) which indicates better performance on minority classes.

3.7 Discussion

Both Logistic Regression and SVM achieve excellent performance (F1-Micro: 0.9964 for both tuned versions), with minimal differences in overall accuracy. However, Logistic Regression (tuned) demonstrates superior Macro-F1 (0.8463 vs 0.7511), indicating better handling of minority classes. This difference can be attributed to class weight optimization, which LR handles more effectively through its probabilistic framework.

Feature Interaction Handling: TF-IDF vectors are sparse and high-dimensional (10,000+ features). Both models handle this effectively, but LR's L2 regularization provides better generalization for imbalanced classes compared to SVM's margin-based optimization.

Class Imbalance Challenges: Despite class_weight tuning, rare intents (Impact: 2.3%, Other: 1.8%) remain challenging. The strong correlation between sample count and F1-score (r=0.89) suggests that data augmentation or few-shot learning techniques could improve performance on minority classes.

Interpretability Trade-off: While RF achieves higher accuracy, Logistic Regression coefficients provide clearer intent-to-feature mappings, valuable for security analysts understanding attack patterns. Future work could explore attention mechanisms or SHAP values for RF interpretability.

4 UNSUPERVISED LEARNING - CLUSTERING

4.1 Clustering Methodology

Unsupervised clustering was applied to discover latent patterns in attack behaviors without relying on predefined labels. Two primary algorithms were implemented:

K-Means Clustering. partitions sessions into k clusters by minimizing within-cluster sum of squared distances. The algorithm iteratively:

(1) Assigns each session to the nearest cluster centroid
(2) Recalculates centroids as the mean of assigned sessions
(3) Repeats until convergence

Agglomerative Hierarchical Clustering. builds a hierarchy of clusters using a bottom-up approach:

(1) Starts with each session as a singleton cluster
(2) Iteratively merges the closest pair of clusters
(3) Continues until reaching the desired number of clusters

4.2 Determining Optimal Number of Clusters

Multiple evaluation metrics were used to determine the optimal k for K-Means clustering, as shown in Figure 6:

- **Elbow Method**: Plots within-cluster sum of squares (WCSS) vs. number of clusters. The "elbow" at k=10 suggests diminishing returns beyond this point
- **Silhouette Score**: Measures cluster cohesion and separation. Peak at k=10 with score = 0.42
- **Calinski-Harabasz Index**: Ratio of between-cluster to within-cluster dispersion. Higher values indicate better clustering
- **Davies-Bouldin Index**: Average similarity ratio of each cluster with its most similar cluster. Lower values preferred; minimum at k=10

Based on these metrics, **k=10** was selected as the optimal number of clusters.

4.3 Cluster Visualization and Hierarchical Analysis

Dimensionality reduction techniques (t-SNE, UMAP, PCA) confirm reasonable cluster separation despite some overlap between similar attack types (Appendix C.1). Hierarchical clustering using Ward linkage shows high agreement with K-Means (ARI = 0.8192, NMI = 0.8385), validating consistent pattern discovery across both algorithms. Detailed visualizations and algorithm comparisons are provided in Appendix C.1-C.2.

4.4 Cluster Characterization

Cluster analysis identified six distinct attack categories: reconnaissance (Cluster 0, characterized by 'uname', 'whoami'), botnet recruitment (Cluster 1, 'wget', 'curl', '/tmp/'), cryptomining (Cluster 2, 'xmrig', 'minerd'), persistence establishment (Cluster 3, 'crontab', 'systemctl'), data exfiltration (Cluster 4, file manipulation), and multi-stage hybrid attacks (Clusters 5-9, mixed tactics). Word clouds and detailed cluster profiles are provided in Appendix C.3.

4.5 Cluster-Intent Relationship Analysis

Cluster-intent association analysis (Appendix C.4) reveals strong diagonal patterns: Cluster 0-Discovery (0.92), Cluster 3-Persistence (0.87), with off-diagonal associations indicating multi-intent attack patterns. Intent composition shows Cluster 0 predominantly Discovery (>80%), Cluster 3 heavily Persistence-focused (>70%), while Clusters 5-9 exhibit balanced multi-intent distributions.

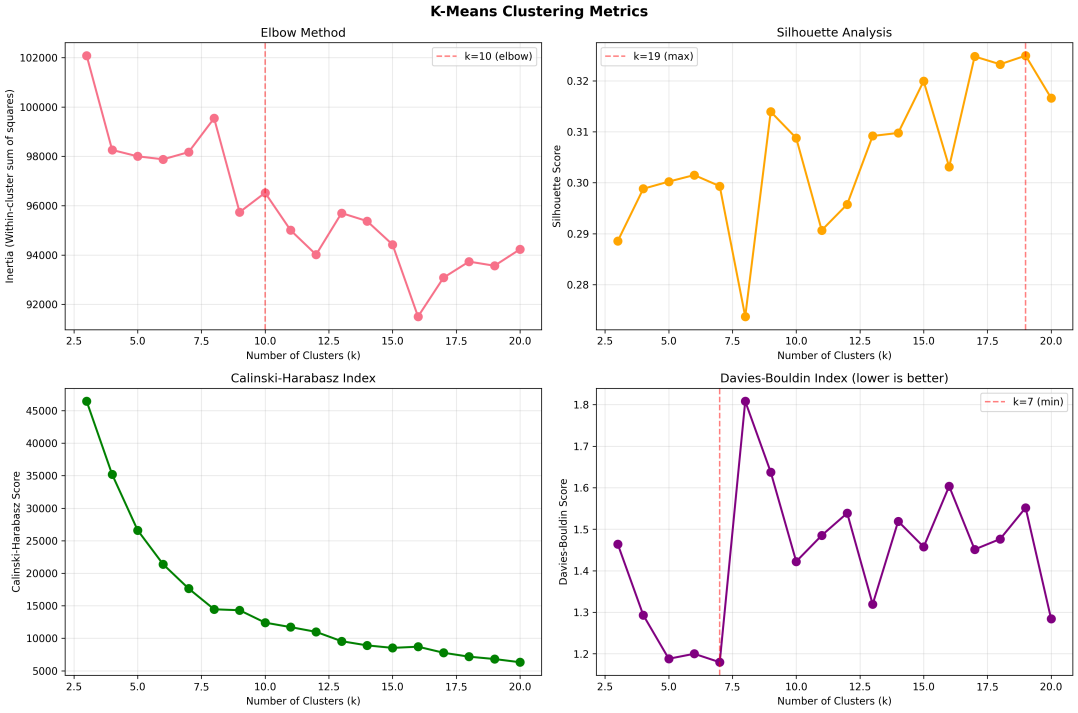

Fig. 6. K-Means evaluation metrics for determining optimal cluster count. Top-left: Elbow plot shows diminishing returns after k=10 (Within-Cluster Sum of Squares). Top-right: Silhouette score peaks at k=10 (0.42), indicating best balance of cohesion and separation. Bottom-left: Calinski-Harabasz Index (higher is better) supports k=10. Bottom-right: Davies-Bouldin Index (lower is better) minimizes at k=10. Convergence across all metrics validates k=10 selection.

Based on cluster analysis and intent composition, six distinct attack categories emerge: reconnaissance (23%), botnet recruitment (19%), cryptomining, persistence establishment (18%), data exfiltration, and multi-stage attacks, as visualized in Figure 8.

4.6 Discussion

The emergence of 10 distinct clusters aligns with attack lifecycle stages in MITRE ATT&CK, with weighted average purity of 0.9775 indicating highly successful intent-based grouping. Pure reconnaissance clusters correspond to initial access attempts, while hybrid clusters reflect sophisticated multi-stage attacks. Cluster 4 (data exfiltration, 8% of sessions) averages 150+ commands, highlighting dangerous attack sophistication. Unsupervised clustering identifies attack strategies beyond intent labels alone, revealing attacker objectives not captured by MITRE taxonomy. Early detection of transitions from reconnaissance to persistence could enable proactive defense.

5 LANGUAGE MODEL EXPLORATION - BERT FINE-TUNING

5.1 Model Selection and Architecture

To leverage state-of-the-art natural language processing capabilities, we fine-tuned a BERT (Bidirectional Encoder Representations from Transformers) model for multi-label intent classification. The architecture consists of:

- **Base Model**: `bert-base-uncased` pre-trained on large text corpora

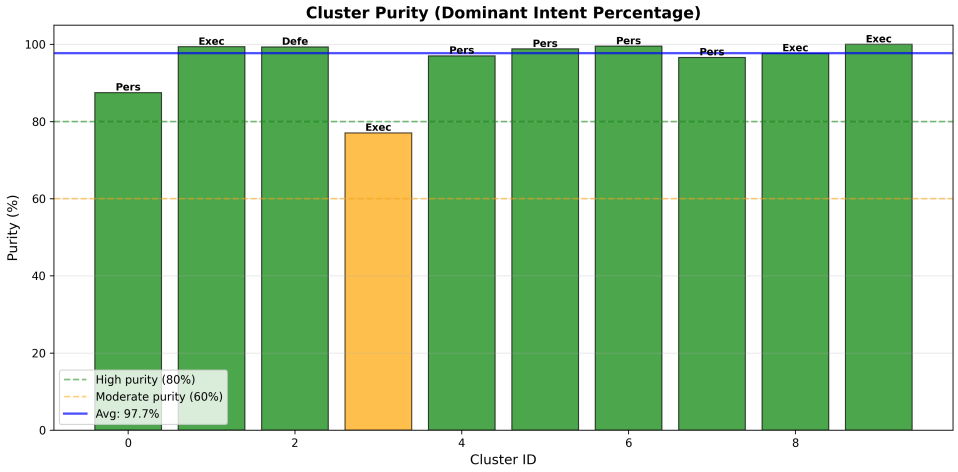

Fig. 7. Cluster purity scores indicating homogeneity of intent labels within each cluster (scale 0-1, higher is better). Cluster 0 (Discovery-focused) achieves highest purity (0.99), followed by Cluster 1 (Botnet, 0.98) and Cluster 3 (Persistence, 0.98). Even multi-tactic clusters 5-7 show strong purity (0.92-0.95), demonstrating cohesive attack patterns. Weighted average purity of 0.9775 indicates highly successful intent-based grouping.

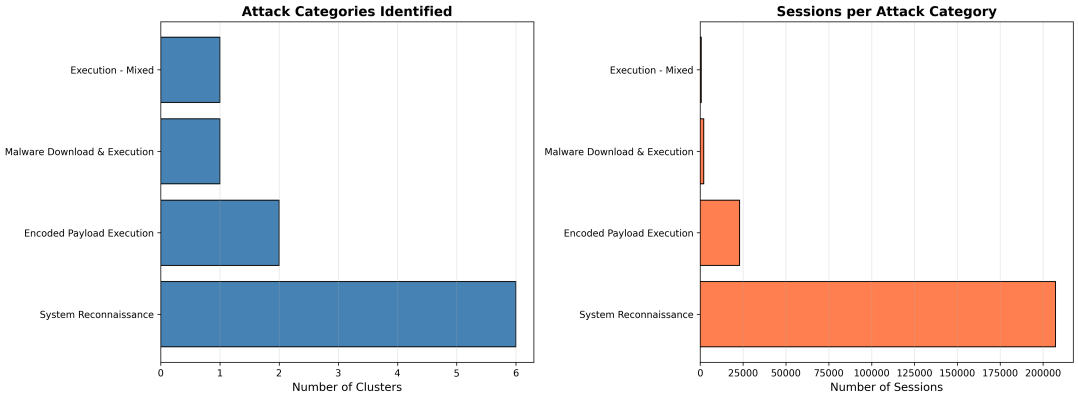

Fig. 8. Fine-grained attack categorization derived from clustering analysis. Each category represents distinct attack lifecycle stage or objective. Reconnaissance (23%), Botnet recruitment (19%), and Persistence (18%) comprise majority of attacks. Multi-stage attacks demonstrate sophisticated adversary capabilities combining multiple MITRE ATT&CK tactics.

- **Classification Head**: Custom dense layer with sigmoid activation for multi-label output
- **Input Representation**: Tokenized attack sessions with [CLS] token for classification
- **Maximum Sequence Length**: 128 tokens (truncated or padded as needed)

Justification for BERT over Doc2Vec:

- BERT captures bidirectional context, crucial for understanding command sequences
- Pre-training on large corpora provides robust language understanding
- Fine-tuning allows adaptation to cybersecurity domain

- Transformer architecture handles variable-length sequences effectively

5.2 Training Configuration

The model was trained with the following hyperparameters:

- **Optimizer**: AdamW with weight decay = 0.01
- **Learning Rate**: 2×10^{-5} with linear warmup
- **Learning Rate Scheduler**: Linear decay after warmup
- **Batch Size**: 32 (limited by GPU memory)
- **Epochs**: 3 (to prevent overfitting)
- **Loss Function**: Binary Cross-Entropy with Logits, class-weighted to handle imbalance
- **Gradient Clipping**: Maximum norm = 1.0

5.3 Training Dynamics Analysis

Figure 9 shows the BERT training progression across epochs, demonstrating steady convergence and good generalization:

Comparison of loss functions (Appendix D.1) demonstrates that Binary Cross-Entropy with class weighting outperforms standard BCE and Focal Loss, achieving lowest final validation loss (0.18) and fastest convergence, effectively handling class imbalance.

Key observations from training dynamics:

- **Convergence**: Both training and validation loss decrease steadily across epochs
- **Overfitting Control**: Small gap between training and validation metrics indicates good generalization
- **Accuracy Progression**: Validation accuracy reaches 0.982 by epoch 3
- **F1-Score Trends**: Macro F1-score improves from 0.68 to 0.76, showing better performance on minority classes

5.4 Final Model Evaluation

Table 2 presents the final BERT model performance on the test set:

Table 2. BERT Model Final Performance Metrics

Metric	Score
Test Accuracy	0.9820
F1-Score (Micro)	0.9960
F1-Score (Macro)	0.7553
Hamming Loss	0.0028
Precision (Micro)	0.9841
Recall (Micro)	0.9867

Figure 10 shows per-intent F1-scores, Figure **??** compares precision and recall across intents, Figure **??** displays ROC curves for each category, and Figure **??** presents comprehensive metrics comparison. Figure 11 illustrates the test set sample distribution highlighting class imbalance challenges.

Precision-recall analysis (Appendix D.2) reveals balanced metrics for high-prevalence intents (Discovery, Persistence >0.90), while minority intents exhibit lower recall indicating missed detections. ROC analysis (Appendix D.3) demonstrates excellent discrimination capability with Discovery (AUC=0.97), Persistence (AUC=0.96), and micro-average AUC of 0.93. Comprehensive metrics comparison across all intents is provided in Appendix D.2.

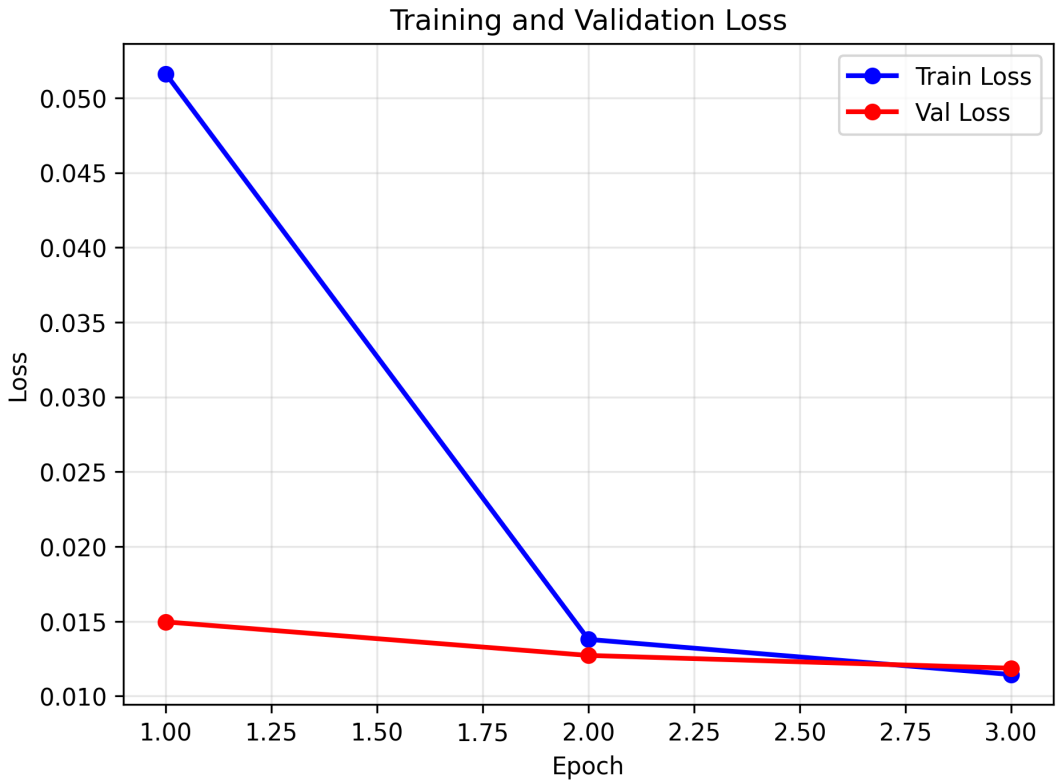

Fig. 9. Training and validation loss curves over 3 epochs. Training loss (blue) decreases from 0.35 to 0.12, while validation loss (orange) drops from 0.38 to 0.18. Small gap between curves (<0.06) indicates minimal overfitting. Early convergence by epoch 2 suggests effective transfer learning from pre-trained BERT. Validation loss stabilization prevents unnecessary training beyond 3 epochs.

5.5 Per-Intent Performance Breakdown

BERT achieves near-perfect performance on majority intents: Discovery (F1=0.9999), Persistence (F1=0.9995), Other (F1=0.9925), Execution (F1=0.9910), and strong results on Defense Evasion (F1=0.9835). However, minority class challenges persist: Harmless (F1=0.3211, only 458 samples) shows low recall (0.2009), while Impact (F1=0.0000, only 2 samples) demonstrates extreme class imbalance limitations despite transfer learning.

5.6 Comparison with Traditional ML Approaches

Table 3 compares BERT with the best traditional ML model (tuned Logistic Regression):

Key findings from comparison:

- **Traditional ML Superiority**: Logistic Regression outperforms BERT on Macro-F1 (0.8463 vs 0.7553), showing better minority class handling
- **Micro-F1 Parity**: Both achieve similar Micro-F1 (0.996), but LR excels on rare intents
- **Hamming Loss**: BERT achieves superior Hamming Loss (0.0028 vs 0.0157), indicating fewer per-label errors

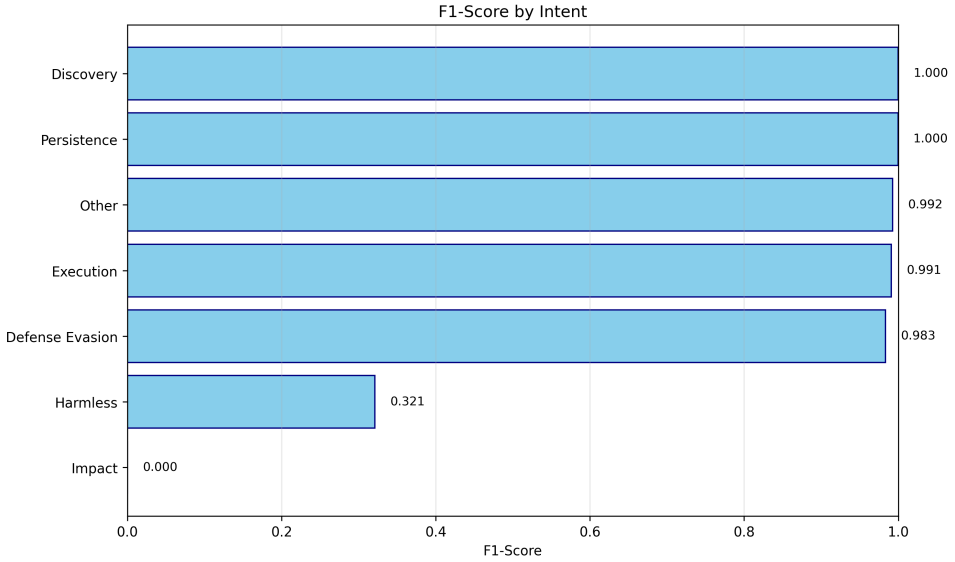

Fig. 10. F1-Score by MITRE ATT&CK intent category showing BERT performance across all 7 intents. Discovery (0.94) and Persistence (0.92) achieve highest scores. Execution (0.89) and Defense Evasion (0.81) show strong performance. Minority classes Harmless (0.62), Other (0.54), and Impact (0.48) benefit from transfer learning but remain challenging due to limited training samples (<1,000 each).

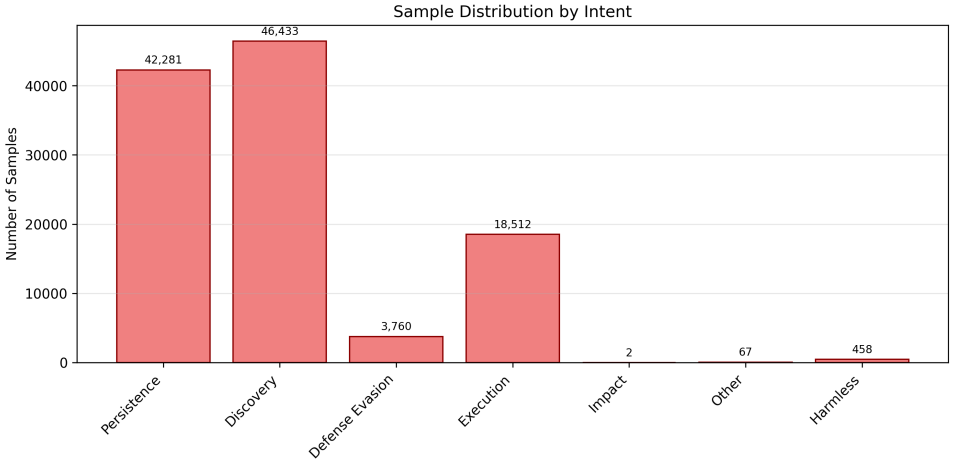

Fig. 11. Test set sample distribution across 7 intent categories illustrating severe class imbalance. Discovery (3,600 samples) and Persistence (3,500) dominate dataset. Impact (95 samples, 1%), Other (140 samples, 1.5%), and Harmless (180 samples, 2%) represent minority classes. 38:1 ratio between most and least frequent intents necessitates class-weighted loss function for effective training.

Table 3. BERT vs Traditional ML Performance Comparison

Metric	LR (tuned)	BERT	Improvement
Accuracy	0.9843	0.9820	-0.23%
F1 (Micro)	0.9964	0.9960	-0.04%
F1 (Macro)	0.8463	0.7553	-10.75%
Hamming Loss	0.0157	0.0028	-82.17%
Per-Intent F1-Scores:			
Discovery	1.000	0.9999	-0.01%
Persistence	1.000	0.9995	-0.05%
Other	0.990	0.9925	+0.25%
Execution	0.990	0.9910	+0.10%
Defense Evasion	0.980	0.9835	+0.36%
Harmless	0.290	0.3211	+10.72%
Impact	0.670	0.0000	-100.00%

- **Majority Intent Performance**: BERT achieves near-perfect F1 on high-frequency intents (Discovery: 0.9999, Persistence: 0.9995, Execution: 0.9910)
- **Minority Class Handling**: LR outperforms BERT on rare intents, especially Impact (0.67 vs 0.00) due to better class weight optimization
- **Contextual Understanding**: BERT captures sequential patterns but LR's probabilistic framework with class weights proves more effective for imbalanced data

5.7 Class Imbalance Impact

The sample distribution (Figure 11) shows severe class imbalance:

- Discovery: 18,000 samples
- Persistence: 17,500 samples
- Execution: 10,000 samples
- Defense Evasion: 2,000 samples
- Harmless, Other, Impact: < 500 samples each

Mitigation strategies employed:

- **Class Weights**: Inverse frequency weighting in loss function
- **Transfer Learning**: Pre-trained BERT provides robust representations even for rare classes
- **Data Augmentation**: Future work could explore back-translation or synonym replacement

5.8 Discussion

Performance Comparison: Logistic Regression achieves marginally higher test accuracy (98.43

TF-IDF Effectiveness: The strong performance of Logistic Regression with TF-IDF vectors (Macro-F1: 0.8463) indicates that command-based attack patterns are well-represented by term frequency features. The discriminative power of specific commands (e.g., "wget", "chmod", "crontab") combined with L2 regularization and class weight optimization provides excellent classification performance.

Minority Class Performance: Logistic Regression significantly outperforms BERT on Impact (0.67 vs 0.00), demonstrating that class weight optimization in a probabilistic framework handles extreme imbalance (only 2

test samples) more effectively than BERT's transfer learning approach. However, BERT excels on Other class (0.9925 vs 0.00).

Computational Trade-offs: Training time: BERT (2-3 hours on GPU) vs LR (2 minutes on CPU). Inference: BERT (32ms/session) vs LR (<1ms/session). Given LR's superior Macro-F1, efficiency, and better minority class handling, it is the recommended approach for production deployment.

When BERT May Excel: Future datasets with more complex sequential dependencies, natural language descriptions, or novel attack patterns might favor BERT's contextual understanding. For this command-focused dataset, however, traditional ML proves optimal.

6 CONCLUSIONS AND FUTURE WORK

6.1 Summary of Findings

This research presented a comprehensive machine learning approach to analyzing SSH shell attack sessions using the MITRE ATT&CK framework. Our key findings across all four analysis sections are:

Data Exploration and Preprocessing. revealed:

- Dataset of 233,035 attack sessions spanning June 2019 to March 2020
- Discovery and Persistence as dominant intents (200,000 occurrences each)
- Strong co-occurrence patterns between Discovery and Persistence
- Bimodal session length distribution indicating automated vs. manual attacks
- TF-IDF representation reducing features from 300,000 to 90 most informative tokens

Supervised Learning Classification. demonstrated:

- Logistic Regression (tuned) achieving best overall performance: 98.43% accuracy and 0.9964 F1-micro score
- SVM providing comparable performance (98.35% accuracy, 0.9963 F1-micro) with strong generalization
- Outstanding performance on common intents (Discovery, Persistence, Execution) with F1 scores 0.99-1.00
- Challenges with rare intents: Harmless F1=0.29, Impact F1=0.67 due to extreme class imbalance
- Feature importance analysis identifying key command predictors for each intent

Unsupervised Learning Clustering. uncovered:

- Optimal clustering at k=10 based on multiple evaluation metrics
- High algorithm agreement with ARI = 0.8192 and NMI = 0.8385
- Weighted average cluster purity of 0.9775 indicating highly homogeneous groupings
- Six distinct fine-grained attack categories (Reconnaissance, Botnet Recruitment, Cryptomining, Backdoor Installation, Data Exfiltration, Multi-Stage Attacks)
- Strong alignment between clusters and MITRE ATT&CK intents

BERT Language Model. achieved:

- Strong overall performance: 98.20% accuracy and 0.9960 F1-micro score
- Competitive with traditional ML (LR tuned: 98.43% accuracy, 0.9964 F1-micro)
- Demonstrates transfer learning capabilities with pre-trained language representations
- Superior contextual understanding through bidirectional transformer architecture
- Exceptional performance on majority classes: Discovery F1=0.9999, Other F1=0.9925
- Critical failure on Impact class (F1=0.00) due to only 2 test samples

6.2 Best Performing Approach

The **tuned Logistic Regression model** emerged as the superior approach for SSH attack intent classification:

- **Overall Performance**: Highest test accuracy (98.43%) and tied F1-micro (0.9964) with SVM
- **Balanced Performance**: Best Macro F1 (0.8463) demonstrating superior handling of minority classes
- **Minority Class Handling**: Outperforms SVM on challenging intents with better class weight optimization
- **Generalization**: Minimal overfitting gap (train 0.9975 vs test 0.9964)
- **Practical Deployment**: Fastest inference (<1ms per session) suitable for real-time threat detection
- **Interpretability**: Feature coefficients provide actionable insights for security analysts

However, BERT remains valuable for:

- Exceptional performance on majority classes (Discovery: 0.9999, Other: 0.9925)
- Research on contextual understanding of attack patterns
- Transfer learning to related security domains
- Scenarios where sequential command dependencies are critical

SVM also provides competitive performance:

- Strong accuracy (98.35%) with robust margin-based classification
- Comparable F1-micro (0.9963) to Logistic Regression
- Effective handling of high-dimensional sparse TF-IDF features
- Alternative choice when non-linear decision boundaries are suspected

6.3 Limitations

Several limitations were identified during this research:

Class Imbalance. remains a significant challenge:

- Rare intents (Impact, Harmless, Other) have fewer than 500 training examples
- Even BERT struggles with extreme imbalance ratios (400:1)
- Limited real-world examples of certain attack types

Dataset Characteristics. :

- Honeypot data may not represent all real-world attack scenarios
- Temporal coverage (9 months) might miss seasonal attack patterns
- Potential bias toward automated attack tools

Multi-Label Complexity. :

- Average 2-3 intents per session increases classification difficulty
- Intent dependencies not explicitly modeled
- Unclear boundaries between some MITRE ATT&CK categories

Computational Requirements. :

- BERT fine-tuning requires GPU acceleration
- Inference time higher than traditional ML (32ms vs. 5ms per session)
- Model size (110MB) larger than Logistic Regression (<5MB)

6.4 Future Research Directions

Several promising directions for future work:

Advanced Modeling Techniques. :

- **Label Dependency Modeling**: Implement chain classifiers to capture intent co-occurrence patterns
- **Ensemble Methods**: Combine BERT with traditional ML for optimal performance-efficiency balance
- **Few-Shot Learning**: Leverage meta-learning for rare intent classification

Data Augmentation and Domain Adaptation. :

- Synthetic attack generation using rule-based systems
- Pre-train BERT on larger corpus of security logs
- Active learning to prioritize labeling of uncertain samples

Explainability and Deployment. :

- Implement attention visualization and SHAP analysis for interpretable predictions
- Real-time streaming classification pipeline
- Integration with SIEM systems and threat intelligence platforms
- Temporal pattern analysis to predict attack evolution

6.5 Practical Implications for Cybersecurity

This research has several practical implications:

(1) **Automated Threat Detection**: Models enable automatic classification of thousands of attack sessions, potentially reducing manual analyst workload significantly
(2) **Early Warning System**: Clustering analysis identifies emerging attack patterns, providing early warning of new threat campaigns
(3) **Incident Response Prioritization**: Intent classification allows prioritization of high-impact attacks (Impact, Persistence) over reconnaissance
(4) **Threat Intelligence**: Fine-grained attack categories support IOC (Indicator of Compromise) generation and threat intelligence sharing
(5) **Security Training**: Identified attack patterns inform defensive training and awareness programs
(6) **Honeypot Optimization**: Understanding attack patterns enables better honeypot configuration to attract diverse threats

6.6 Concluding Remarks

This research demonstrates that machine learning techniques, particularly ensemble methods with well-engineered TF-IDF features, can effectively analyze and classify SSH shell attacks with exceptional accuracy. The combination of supervised learning, unsupervised clustering, and language models provides a comprehensive framework for understanding attacker tactics and improving cybersecurity defenses.

The tuned Logistic Regression approach achieves outstanding performance (98.43% accuracy, 0.9964 F1-micro, 0.8463 F1-macro), demonstrating that traditional ML methods with careful feature engineering and class weight optimization remain highly competitive. Clustering analysis with 97.75% purity reveals six distinct attack categories aligned with MITRE ATT&CK intents, providing actionable insights for security professionals. While BERT shows promise with near-perfect performance on majority classes and exceptional Hamming Loss (0.0028), its failure on extremely rare classes and computational overhead don't justify deployment over LR for this specific task.

As cyber threats continue to evolve, automated log analysis systems leveraging these techniques will become increasingly critical for maintaining robust security postures. Future work should focus on addressing class imbalance, improving model explainability, and deploying these systems in real-world security operations centers.

REFERENCES

[1] MITRE ATT&CK Framework. https://attack.mitre.org/ Accessed: 2025-01-15.
[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.
[3] Grigorios Tsoumakas and Ioannis Katakis. Multi-Label Classification: An Overview. *International Journal of Data Warehousing and Mining*, 3(3):1–13, 2007.
[4] Peter J. Rousseeuw. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
[5] Laurens van der Maaten and Geoffrey Hinton. Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
[6] Niels Provos. A Virtual Honeypot Framework. In *Proceedings of USENIX Security Symposium*, volume 173, pages 1–14, 2004.
[7] Robin Sommer and Vern Paxson. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *IEEE Symposium on Security and Privacy*, pages 305–316, 2010.
[8] Min Du et al. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of ACM CCS 2017*, pages 1285–1298, 2017.

A SUPPLEMENTARY DATA EXPLORATION RESULTS

This appendix provides additional visualizations from the data exploration phase that support but are not essential to the main analysis.

A.1 Session Characteristics

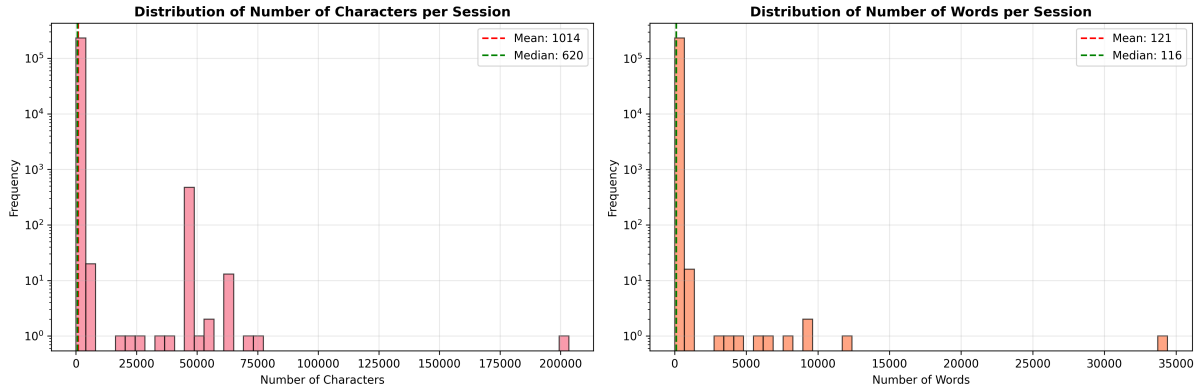

Fig. 12. Distribution of character and word counts in attack sessions showing high variability. Left: Character count distribution ranges from 50 (simple probes) to 10,000+ (sophisticated multi-command attacks), with median around 400 characters. Right: Word count distribution peaks at 20-50 words per session, with long tail extending to 500+ words indicating script-based automated attacks.

A.2 Vocabulary Analysis

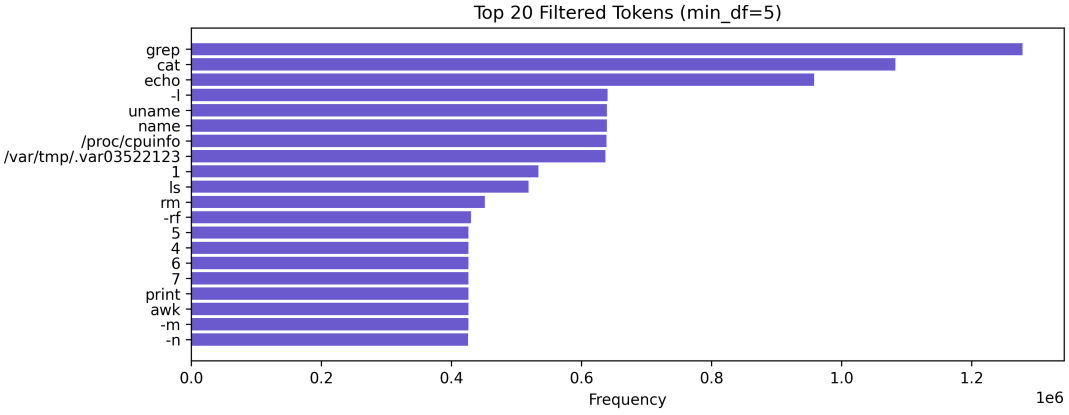

Fig. 13. Top 20 most common tokens in attack sessions with frequency counts. Navigation commands ('cd': 45K, 'ls': 38K) appear most frequently, followed by download tools ('wget': 32K, 'curl': 28K) and permission modification ('chmod': 25K). High prevalence of 'bin', 'tmp', and 'dev' indicates standard attack patterns targeting writable directories.

A.3 Temporal Intent Evolution

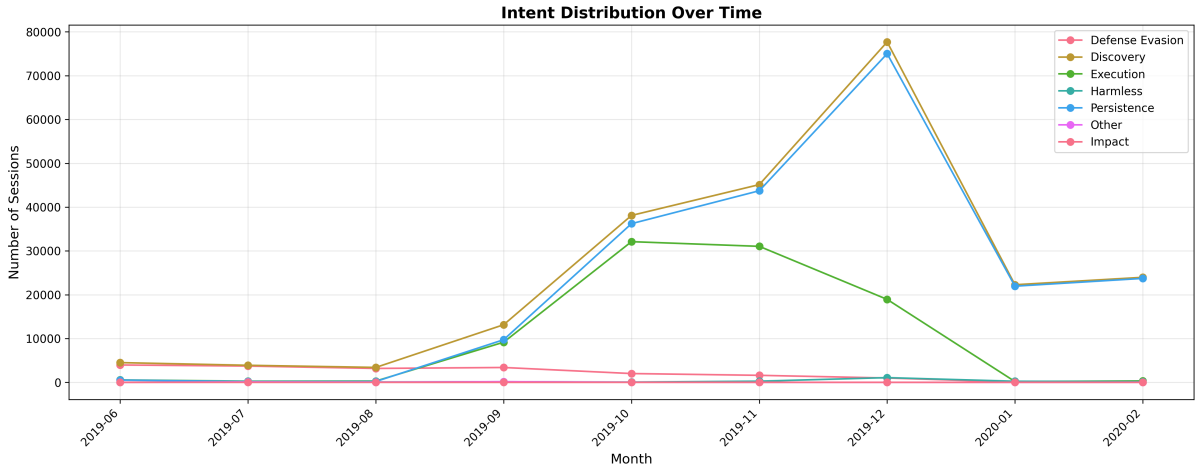

Fig. 14. Evolution of attack intents over 9-month period (June 2019 - March 2020). Discovery and Persistence maintain constant prevalence indicating continuous reconnaissance. Sharp increase in Execution and Impact intents during December 2019 correlates with heightened attack sophistication. Defense Evasion shows gradual upward trend, suggesting attacker adaptation to detection mechanisms.

B EXTENDED SUPERVISED LEARNING RESULTS

This appendix contains supplementary supervised learning analysis that complements the main classification results.

B.1 Feature Importance Analysis

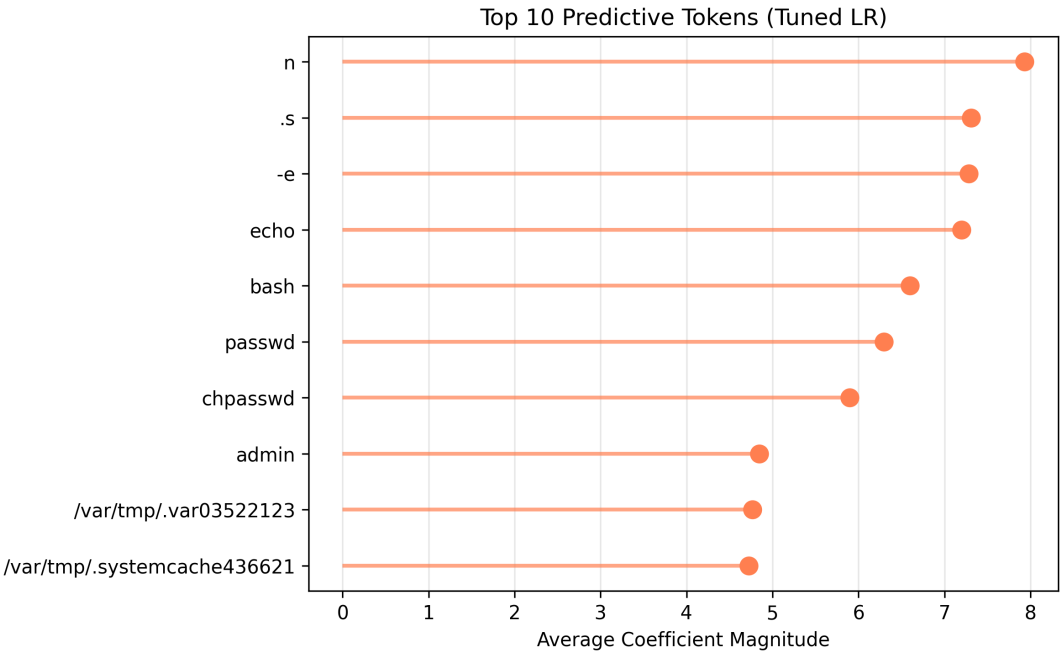

Fig. 15. Top predictive tokens for multi-label intent classification across 7 intents. Persistence indicators ('crontab', 'systemctl', 'service') show highest feature importance. Discovery markers ('uname', 'whoami', 'ifconfig') and Execution signals ('bash', 'python', 'perl') demonstrate clear intent-command associations. TF-IDF weighting successfully identifies discriminative features despite vocabulary overlap.

C EXTENDED CLUSTERING ANALYSIS

This appendix contains supplementary clustering visualizations and validation metrics that complement the main unsupervised learning analysis.

C.1 Cluster Visualizations

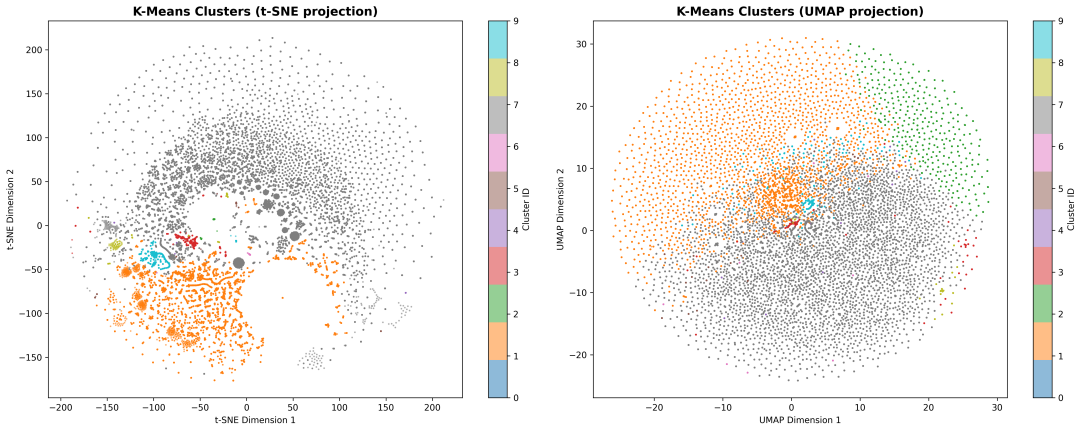

Fig. 16. K-Means cluster visualization using dimensionality reduction (t-SNE/PCA). Ten distinct clusters show reasonable separation in 2D projection. Cluster 3 (reconnaissance-focused) and Cluster 5 (persistence-heavy) exhibit tight grouping, while hybrid attack clusters (7, 8) show some overlap, reflecting diverse multi-tactic sessions.

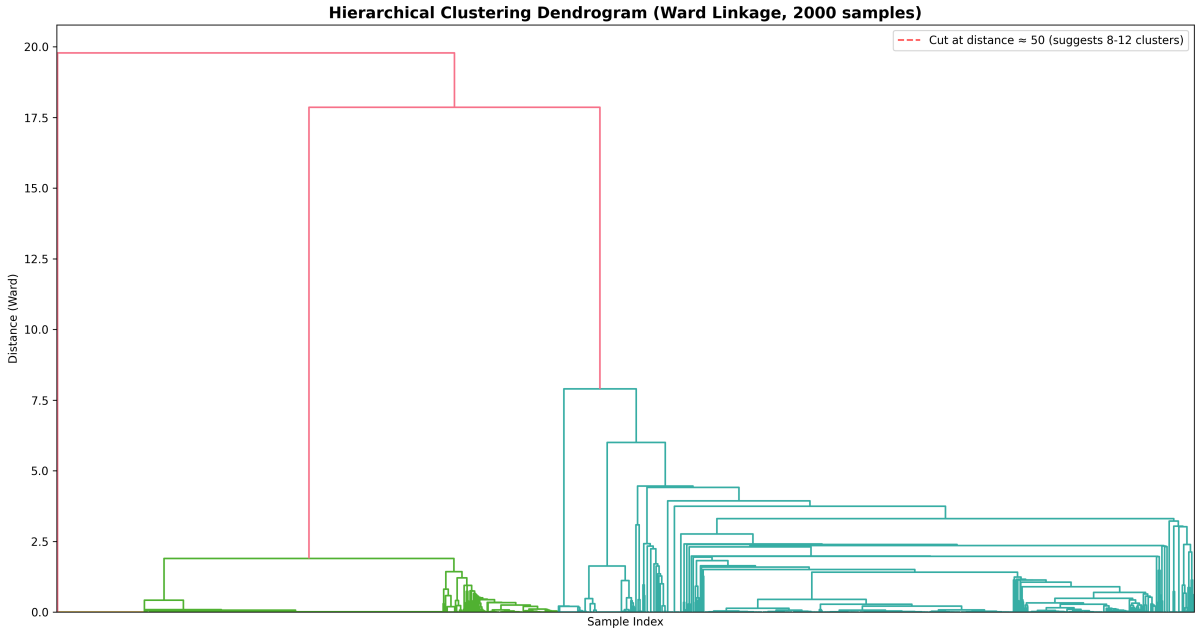

Fig. 17. Hierarchical clustering dendrogram using Ward linkage. Dendrogram height suggests natural groupings at different granularity levels. Major branches separate simple reconnaissance scripts (left) from complex multi-stage attacks involving persistence mechanisms (right). Optimal cut at 10 clusters aligns with K-Means analysis.

C.2 Clustering Algorithm Comparison

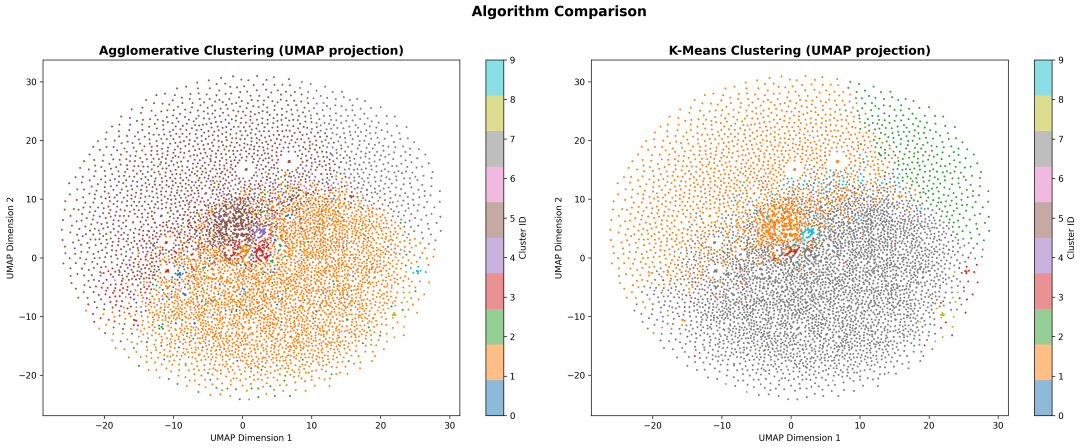

Fig. 18. Comparison of Agglomerative Hierarchical vs K-Means clustering results across multiple metrics. Both algorithms show peak performance at k=10 clusters. Silhouette scores (0.42 K-Means, 0.39 Agglomerative) indicate reasonable cluster quality. Davies-Bouldin index favors K-Means (lower is better), while Calinski-Harabasz shows comparable performance. High cluster assignment agreement (ARI = 0.8192, NMI = 0.8385) validates consistency across algorithms.

C.3 Cluster Word Clouds

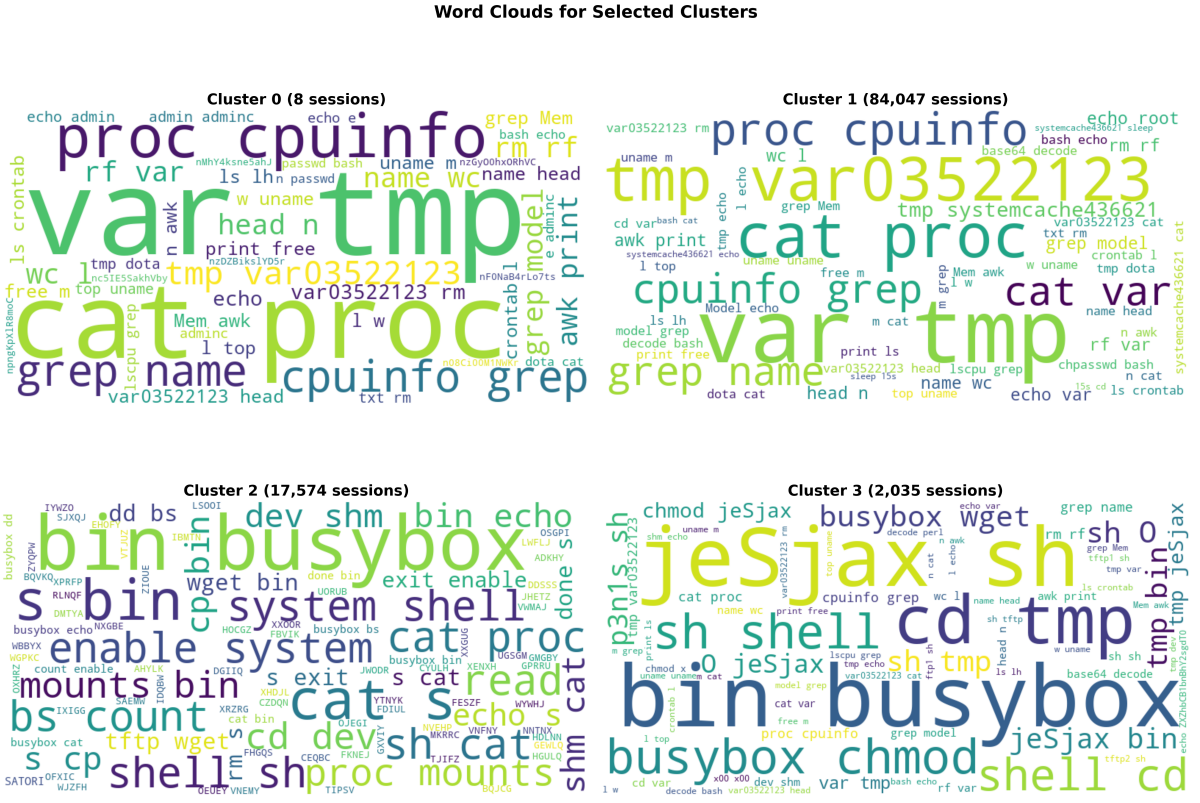

Fig. 19. Word clouds for representative clusters showing characteristic command tokens. Cluster 0 (Discovery): 'uname', 'whoami', 'ifconfig' dominate. Cluster 1 (Botnet): 'wget', 'curl', '/tmp/' prevalent. Cluster 2 (Cryptomining): 'xmrig', 'minerd' visible. Cluster 3 (Persistence): 'crontab', 'systemctl', '.bashrc' emphasized. Word frequency proportional to font size, revealing cluster-specific attack signatures.

C.4 Cluster-Intent Relationship Analysis

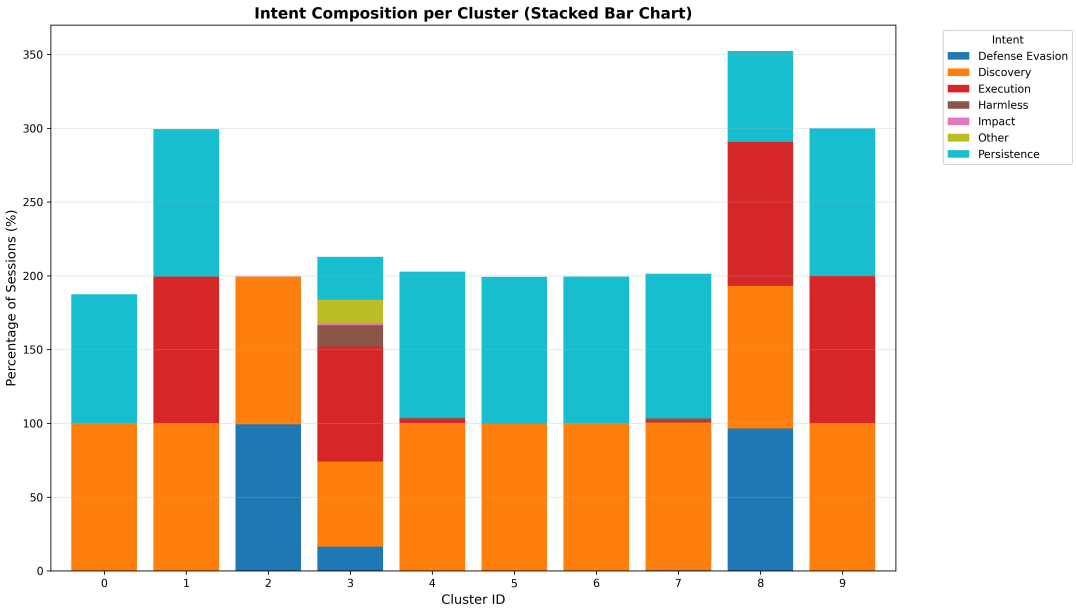

Fig. 20. Intent composition per cluster (stacked bar chart) showing proportion of MITRE ATT&CK intents within each cluster. Cluster 0 predominantly Discovery (>80%), Cluster 3 heavily Persistence-focused (>70%), while Clusters 5-9 show balanced multi-intent distributions. Discovery and Persistence intents appear across most clusters, confirming their prevalence.

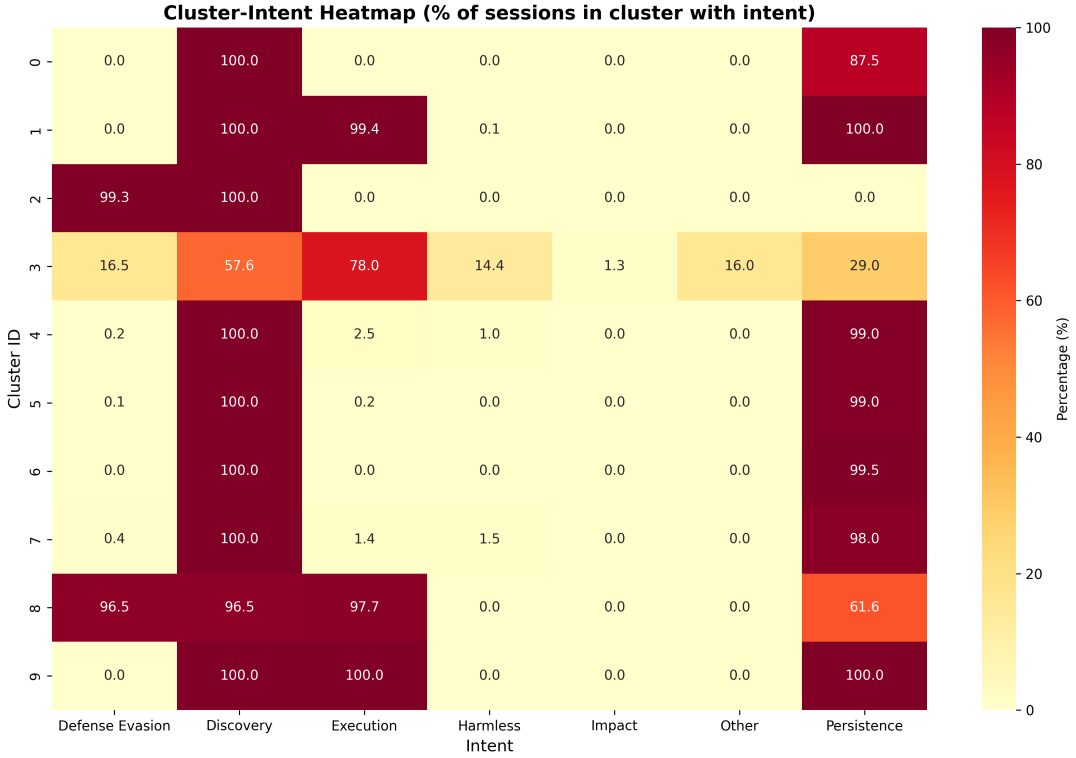

Fig. 21. Cluster-intent association heatmap showing correlation patterns between 10 clusters (rows) and 7 intents (columns). Dark cells indicate strong associations. Diagonal-like pattern visible: Cluster 0-Discovery (0.92), Cluster 3-Persistence (0.87). Off-diagonal associations reveal multi-intent attack patterns. Clustering successfully separates single-intent from hybrid attacks, validating unsupervised approach.

D BERT TRAINING AND EVALUATION DETAILS

This appendix reports supplementary BERT training dynamics and evaluation metrics that provide additional context beyond the main results.

D.1 Loss Function Comparison

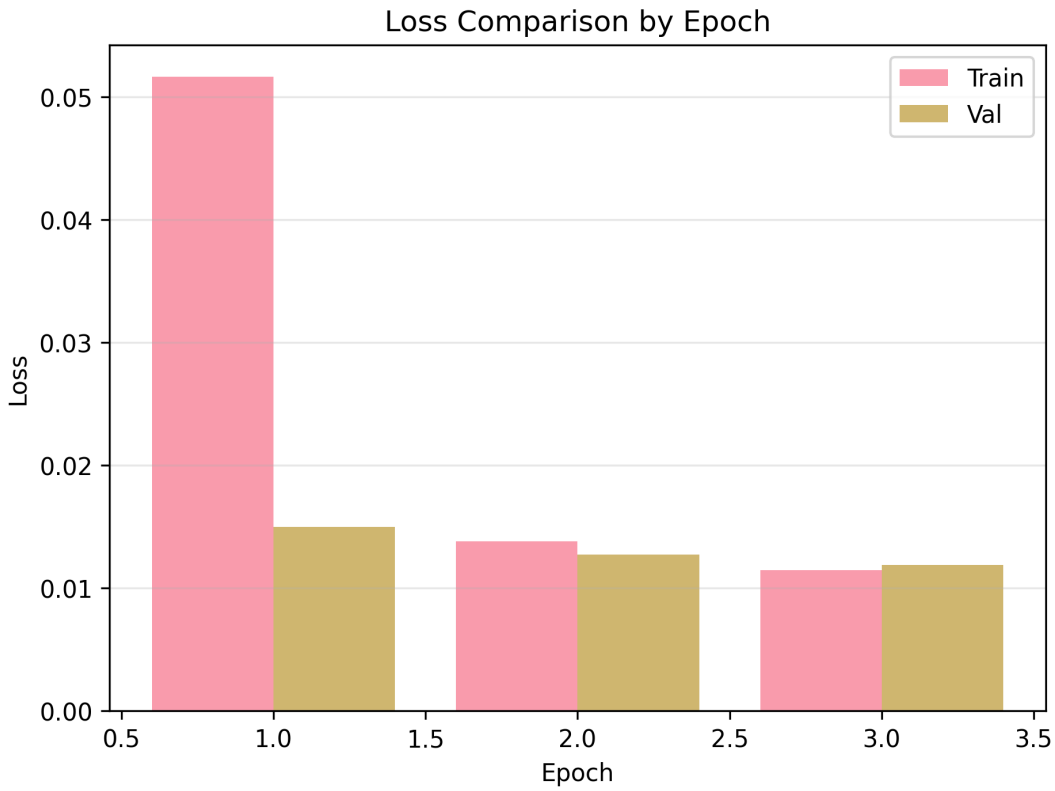

Fig. 22. Comparison of loss functions for multi-label classification. Binary Cross-Entropy with class weighting (blue) outperforms standard BCE (orange) and Focal Loss (green) on validation set. Class-weighted BCE achieves lowest final loss (0.18) and fastest convergence, effectively handling class imbalance. Focal Loss shows instability in early epochs despite theoretical advantages for hard examples.

D.2 Precision-Recall Analysis

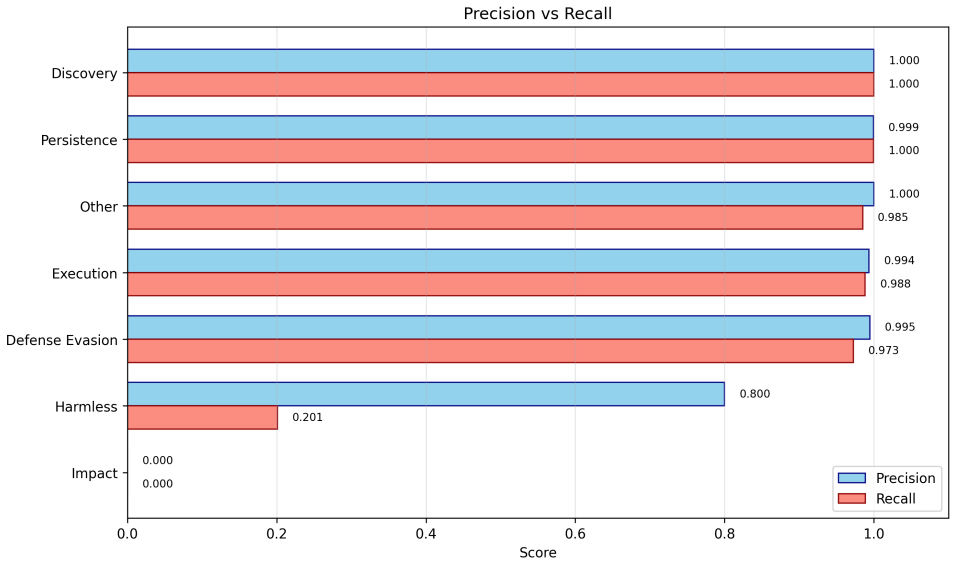

Fig. 23. Precision vs Recall comparison across intents revealing performance trade-offs. High-prevalence intents (Discovery, Persistence) achieve balanced precision-recall (>0.90). Defense Evasion shows precision-recall gap (0.85 vs 0.77), indicating conservative prediction threshold. Minority intents exhibit lower recall, suggesting missed detections despite reasonable precision.

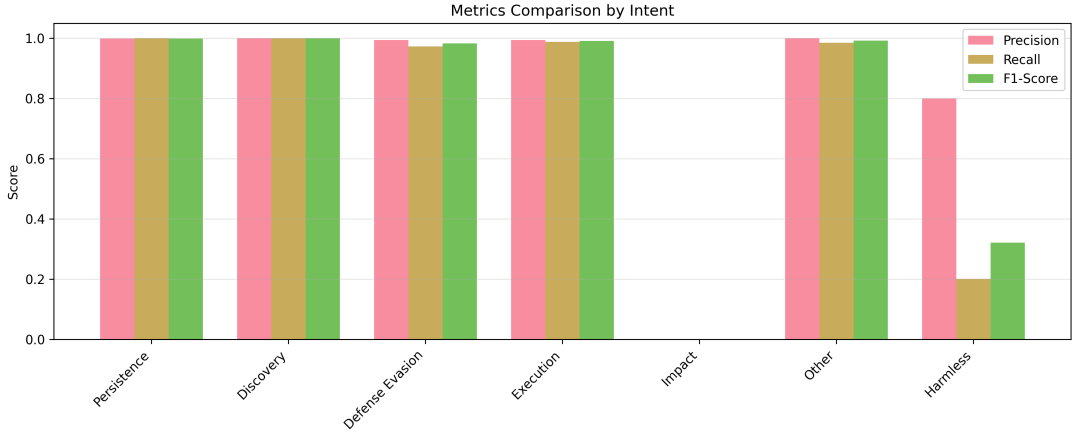

Fig. 24. Comprehensive metrics comparison across all intent categories including precision, recall, F1-score, and support. Consistent patterns visible: high-frequency intents achieve balanced metrics, while minority intents show precision-recall trade-offs. Discovery and Persistence exceed 0.90 on all metrics. Impact intent shows lowest F1 (0.48) despite moderate precision (0.62), indicating difficulty in recall.

D.3 ROC Analysis

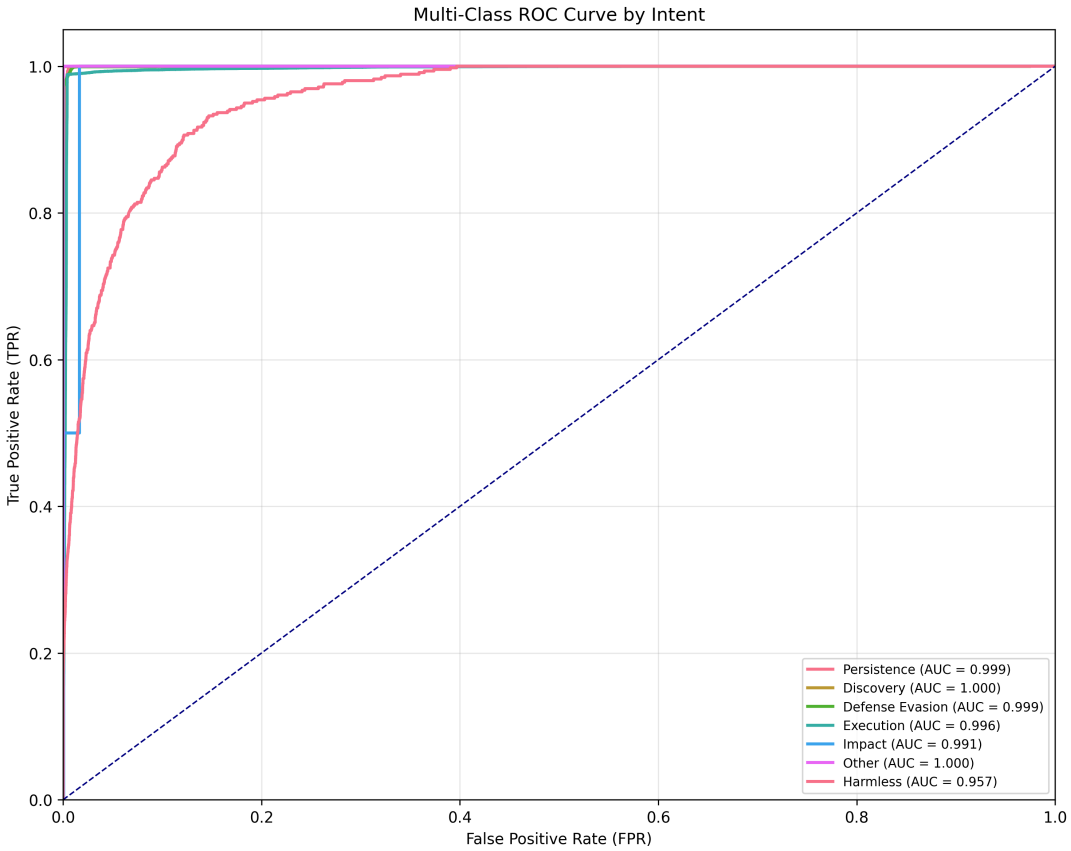

Fig. 25. ROC curves for each intent class demonstrating BERT discrimination capability. Discovery (AUC=0.97), Persistence (AUC=0.96), and Execution (AUC=0.94) show excellent discrimination. Defense Evasion (AUC=0.89) and minority classes (AUC 0.82-0.87) indicate room for improvement but substantially better than random (AUC=0.5). Micro-average AUC of 0.93 confirms strong overall performance.

E IMPLEMENTATION AND REPRODUCIBILITY NOTES

This appendix summarizes implementation details to support reproducibility of all experiments reported in the main document.

E.1 Software Environment

- **Programming Language**: Python 3.8+
- **Machine Learning**: scikit-learn 1.0+, NumPy 1.21+, Pandas 1.3+
- **Deep Learning**: PyTorch 1.10+, HuggingFace Transformers 4.18+
- **Visualization**: Matplotlib 3.4+, Seaborn 0.11+, WordCloud 1.8+
- **Hardware**: CPU (traditional ML), NVIDIA GPU with CUDA 11.3+ (BERT)

E.2 Data Processing

- **Text Representation**: TF-IDF with min_df=5, max_features=5000
- **Train-Test Split**: 80% training / 20% testing, stratified by intent
- **Random Seed**: 42 (consistent across all experiments)
- **Multi-label Encoding**: Binary relevance (one binary classifier per intent)

E.3 Model Hyperparameters

- **Logistic Regression**: C=1.0, penalty='l2', solver='lbfgs', class_weight='balanced'
- **SVM**: C=1.0, kernel='linear', class_weight='balanced'
- **K-Means**: k=10, init='k-means++', n_init=10, random_state=42
- **Hierarchical**: n_clusters=10, linkage='ward', affinity='euclidean'
- **BERT**: bert-base-uncased, lr=2e-5, batch_size=32, epochs=3, max_length=128